# Circuit Trait

halo2에서 `Circuit` trait은 영지식 증명을 위한 산술 회로를 정의하는 핵심 인터페이스입니다. 모든 halo2 회로는 이 trait을 구현해야 하며, 회로의 구조와 동작을 정의합니다.

## Circuit Trait 구조

```
pub trait Circuit<F: Field> {
type Config: Clone + Debug;
type FloorPlanner: FloorPlanner;

fn without_witnesses(&self) -> Self;
fn configure(meta: &mut ConstraintSystem<F>) -> Self::Config;
fn synthesize(
&self,
config: Self::Config,
layouter: impl Layouter<F>,
) -> Result<(), plonk::Error>;
}
```

### 연관 타입 (Associated Types)

#### Config

- **목적**: 회로의 설정 정보를 저장하는 타입
- **요구사항**: `Clone + Debug` trait을 구현해야 함
- **역할**: 회로의 제약조건, 컬럼 정의, 게이트 설정 등을 포함

**FloorPlanner**

- **목적**: 회로의 물리적 레이아웃 전략을 결정
- **일반적인 선택**: `SimpleFloorPlanner` (대부분의 경우 적합)
- **역할**: 회로 요소들의 배치와 최적화를 담당

# 필수 메서드들

## 1. without_witnesses()

```
fn without_witnesses(&self) -> Self;
```

**목적**: witness 데이터 없이 회로의 구조만을 복사하는 메서드

**사용 시점**:

- Proving Key와 Verifying Key 생성 시
- 회로 구조 검증 시
- 테스트 환경에서 회로 구조 확인 시

**구현 예제**:

```
fn without_witnesses(&self) -> Self {
TestCircuit { _ph: PhantomData }
}
```

**중요 사항**:

- 실제 witness 값들은 제거하고 회로 구조만 유지
- 모든 private input은 기본값 또는 더미 값으로 대체
- public input과 회로 구조는 그대로 유지

## 2. configure()

```
fn configure(meta: &mut ConstraintSystem<F>) -> Self::Config;
```

**목적**: 회로의 제약조건과 구조를 정의하는 메서드

**매개변수**:

- `meta` : 제약조건 시스템에 대한 가변 참조
- 컬럼 생성, 게이트 정의, 제약조건 추가 등에 사용

**주요 작업**:

1. **컬럼 생성**: advice, instance, fixed 컬럼들 정의
2. **게이트 정의**: 커스텀 게이트와 제약조건 생성
3. **설정 반환**: 생성된 설정을 Config 타입으로 반환

**구현 예제**:

```
fn configure(meta: &mut ConstraintSystem<F>) -> Self::Config {
// 컬럼 생성
let advice = meta.advice_column();
let instance = meta.instance_column();
let constant = meta.fixed_column();
```

```
// 등식 제약조건 활성화
meta.enable_equality(advice);
meta.enable_equality(instance);
// 설정 반환
TestConfig {
advice,
instance,
constant,
}
}
```

### 3. synthesize()

```
fn synthesize(
&self,
config: Self::Config,
layouter: impl Layouter<F>,
) -> Result<(), plonk::Error>;
```

**목적**: 실제 witness 값들을 회로에 할당하는 메서드

**매개변수**:

- `config` : configure()에서 생성된 회로 설정
- `layouter` : 회로 레이아웃을 관리하는 도구

**주요 작업**:

1. **Region 생성**: 제약조건이 적용될 영역 정의
2. **값 할당**: witness 값들을 적절한 셀에 할당
3. **제약조건 적용**: 정의된 게이트와 제약조건 활성화

## 실제 구현 예제: NOP Circuit

다음은 아무 작업도 수행하지 않는 가장 간단한 회로 예제입니다:

```
use std::marker::PhantomData;
use halo2_proofs::{
circuit::{Layouter, SimpleFloorPlanner},
plonk::{self, Circuit, ConstraintSystem},
};
use ff::Field;


// 회로 구조체 정의
struct TestCircuit<F: Field> {
_ph: PhantomData<F>, // 타입 매개변수 유지용
}


// 설정 구조체 정의
#[derive(Clone, Debug)]
struct TestConfig<F: Field + Clone> {
_ph: PhantomData<F>,
```

```
}

// Circuit trait 구현
impl<F: Field> Circuit<F> for TestCircuit<F> {
type Config = TestConfig<F>;
type FloorPlanner = SimpleFloorPlanner;

fn without_witnesses(&self) -> Self {
TestCircuit { _ph: PhantomData }
}

fn configure(meta: &mut ConstraintSystem<F>) -> Self::Config {
// 이 예제에서는 제약조건 없음
TestConfig { _ph: PhantomData }
}

fn synthesize(
&self,
config: Self::Config,
layouter: impl Layouter<F>,
) -> Result<(), plonk::Error> {
// 이 예제에서는 값 할당 없음
Ok(())
```

```
}

}
```

## 사용 예제

```
fn main() {

use halo2_proofs::halo2curves::bn256::Fr;

use halo2_proofs::dev::MockProver;

// 회로 인스턴스 생성

let circuit = TestCircuit::<Fr> { _ph: PhantomData };

// MockProver로 회로 테스트

let prover = MockProver::run(8, &circuit, vec![]).unwrap();

prover.verify().unwrap();

println!("회로 검증 성공!");

}
```

## 설계 고려사항

### 1. 타입 안전성

- 제네릭 타입 `F: Field` 를 통해 다양한 유한체 지원
- PhantomData를 사용하여 타입 정보 유지

### 2. 모듈화

- Config 타입을 통해 회로 설정과 로직 분리
- 재사용 가능한 컴포넌트 설계 가능

### 3. 성능 최적화

- without_witnesses()를 통해 불필요한 데이터 복사 방지
- FloorPlanner를 통한 레이아웃 최적화

## 다음 단계

Circuit trait의 기본 구조를 이해했다면, 다음 문서들을 통해 더 자세한 구현 방법을 학습할 수 있습니다:

- Configuration과 Synthesis: configure()와 synthesize() 메서드의 상세 구현
- Chip과 Region: 모듈화된 회로 설계 패턴
- Layouter와 FloorPlanner: 효율적인 회로 레이아웃 전략

Circuit trait은 halo2 회로 설계의 출발점입니다. 이 기본 구조를 바탕으로 복잡한 영지식 증명 시스템을 구축할 수 있습니다.